Practice 11. Topological Sort and Dijkstra's Algorithm

[CSE2010] Data Structures

Department of Data Science

Practice 9 Max Heap Solutions

in Python and C++

```python
# Practice 9. Max Heap
import sys
INSERT = 'I'
DELETE = 'D'
MAXIMUM = 'M'
MAX_CAPACITY = 1000
INT_MIN = -sys.maxsize

class MaxHeap:
  def __init__(self, num=MAX_CAPACITY):
    self.elements = [0] * num
    self.count = 0
    self.capacity = num

  # Given the index i of element,
  # return the index of that element's parent in the heap
  def parent(self, i):
    return (i - 1) // 2;

  # Given the index i of element,
  # return the index of that element's left child in the heap
  def left(self, i):
    return 2 * i + 1

  # Given the index i of element,
  # return the index of that element's right child in the heap
  def right(Self, i):
    return 2 * i + 2

  def isEmpty(self):
    return self.count == 0

  def isFull(self):
    return self.count == self.capacity
```

```python
  def maxHeapify(self, i):
    l, r, maxIndex = self.left(i), self.right(i), i
    if l < self.count and self.elements[l] > self.elements[i]:
      maxIndex = l
    if r < self.count and self.elements[r] > self.elements[maxIndex]:
      maxIndex = r
    if maxIndex != i:
      temp = self.elements[i]
      self.elements[i] = self.elements[maxIndex]
      self.elements[maxIndex] = temp
      self.maxHeapify(maxIndex)
  # Insert a new element into the heap
  # If the insertion fails,
  # immediately terminate the program with the error message.
  def insertElement(self, elem):
    if self.isFull():
      raise Exception("INSERT: heap is full")
    i = self.count
    pi = self.parent(i)
    self.count += 1
    while i > 0 and self.elements[pi] < elem:
      self.elements[i] = self.elements[pi]
      i = pi
      pi = self.parent(i)
    self.elements[i] = elem

  # Return the maximum of the heap if it exists
  # Otherwise, terminate program with an error
  def maximum(self):
    if self.isEmpty():
      raise Exception("MAXIMUM: heap is empty")
    else:
      return self.elements[0]

  # Delete the maximum from the heap and return the maximum
  # If deletion fails, terminate program with an error
  def deleteMaximum(self):
    if self.isEmpty():
      raise Exception("DELETE: heap is empty")
    maxVal = self.elements[0]
    self.elements[0] = self.elements[self.count - 1]
    self.count -= 1
    self.maxHeapify(0)
    return maxVal
```

```cpp
template <class T>
class MaxHeap {
  int capacity;
  int count;
  T* elements;
  // Given the index i of element,
  // return the index of that element's parent in the heap
  int parent(int i) {
    return (i - 1) / 2;
  }
  // Given the index i of element,
  // return the index of that element's left child in the heap
  int left(int i) {
    return 2 * i + 1;
  }
  // Given the index i of element,
  // return the index of that element's right child in the heap
  int right(int i) {
    return 2 * i + 2;
  }
  void maxHeapify(int i);
public:
  MaxHeap(int num = MAX_CAPACITY) {
    elements = new T[num];
    count = 0;
    capacity = num;
  }
  ~MaxHeap() {
    delete elements;
  }
  bool isEmpty() {
    return count == 0;
  }
  bool isFull() {
    return count == capacity;
  }
  // Insert a new element into the heap
  // If the insertion fails,
  // immediately terminate the program with the error message.
  void insertElement(T);
  // Return the maximum of the heap if it exists
  // Otherwise, terminate program with an error
  T maximum();
  // Delete the maximum from the heap and return the maximum
  // If deletion fails, terminate program with an error
  T deleteMaximum();
```

```cpp
template <class T>
void MaxHeap<T>::insertElement(T x) {
  if (isFull()) {
    cerr<<"INSERT: heap is full"<<endl;
    exit(1);
  }
  int i = count;
  int pi = parent(i);
  ++count;
  while (i > 0 && elements[pi] < x) {
    elements[i] = elements[pi];
    i = pi;
    pi = parent(i);
  }
  elements[i] = x;
}

template <class T>
T MaxHeap<T>::maximum() {
  if (isEmpty()) {
    cerr<<"MAXIMUM: heap is empty"<<endl;
    exit(1);
  }
  else
    return elements[0];
}
```

```
template <class T>
void MaxHeap<T>::maxHeapify(int i) {
  int l = left(i), r = right(i), maxIndex = i;
  if (l < count && elements[l] > elements[i])
    maxIndex = l;
  if (r < count && elements[r] > elements[maxIndex])
    maxIndex = r;
  if (maxIndex != i) {
    T temp = elements[i];
    elements[i] = elements[maxIndex];
    elements[maxIndex] = temp;
    maxHeapify(maxIndex);
  }
}
```

```
template <class T>
T MaxHeap<T>::deleteMaximum() {
  if (isEmpty()) {
    cerr<<"DELETE: heap is empty"<<endl;
    exit(1);
  }
  int max = elements[0];
  elements[0] = elements[(count--) - 1];
  maxHeapify(0);
  return max;
}
```

TODO for Today's Practice

- Implement topological sort in a connected and directed **graph**.
  - Runtime: $O(|V|+|E|)$ (where the graph has $|V|$ vertices and $|E|$ edges)
- Implement Dijkstra's algorithm in a connected and directed **graph** with weighted edges.
  - Runtime: $O(|V|\log|V| + |E|\log|V|)$

Input

- Vertices and directed and weighted edges in a **directed graph**.
- **C<space>[number_of_vertices]<space>[number_of_edges]**
  [vertex_id]<space>[vertex_id]<space>[weight]
  ⋮
  [vertex_id]<space>[vertex_id]<space>[weight]
  [number_of_edges]
  - If the construction fails, immediately terminate the program with the error message.
  - **Given n vertices, assume that vertex IDs are given as 0, 1, …, n-1.**
- **T**
  - Write the topological sort into output file.
- **S<space>[source]<space>[target]**
  - write **the shortest path** to the output file: **[vertex_id]<space>…<space>[vertex_id]**

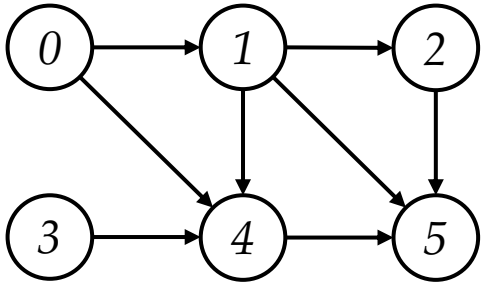
Input and Output Examples

- Output file contains the topological sort.
- Input File

```
C 6 8
0 1 1
0 4 1
1 2 1
1 4 1
1 5 1
2 5 1
3 4 1
4 5 1
T
```

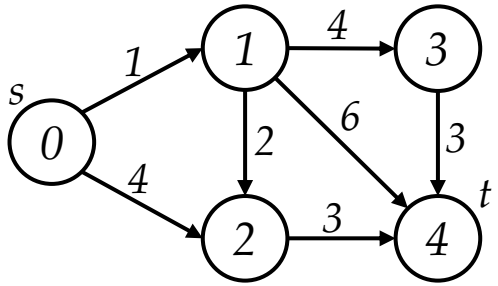
Output File

```
0 3 1 2 4 5
```



Input & Output Examples

- The shortest path from vertex 0 to vertex 4 exists, so path “0 1 2 4” is written into output file.
- Input file




```
C 5 7
0 1 1
0 2 4
1 2 2
1 3 4
1 4 6
2 4 3
3 4 3
S 0 4
```

- Output file

```
0 1 2 4
```

no space at the end of each line in output file

Running Example

- Start from the last practice code.

```
[hjkim@localhost graphs]$ g++ -std=c++11 -o graph graph.cpp
[hjkim@localhost graphs]$ cat input6.txt
C 6 8
0 1 1
0 4 1
1 2 1
1 4 1
1 5 1
2 5 1
3 4 1
4 5 1
T
[hjkim@localhost graphs]$ ./graph input6.txt output.txt
0: (1, 1) (4, 1)
1: (2, 1) (4, 1) (5, 1)
2: (5, 1)
3: (4, 1)
4: (5, 1)
5:
[hjkim@localhost graphs]$ cat output.txt
0 3 1 2 4 5
```

Submission Guideline

- Submission: **source code, an input file, an output file**
  - Where: Assignments in LMS
  - Deadline: **23:59, May. 19th (Sunday)**

Hints

- Python: use deque, which can behave like stack or queue.
  - from collections import deque
- C/C++: use the queue or stack implementations
  - #include <queue>
  - #include <stack>
- Python: use either heapq (min priority queue implementation) or your heap implementation.
  - import heapq
- C/C++: use either queue (max priority queue implementation) or your heap implementation.
  - #include <queue>

Priority Queue Implementation

Python: min priority queue

- `import heapq`
- Initialize
  - `heap = []`
- Insert x
  - `heapq.heappush(heap, x)`
- Delete and return the minimum
  - `heapq.heappop(heap)`
- Get the minimum
  - `heap[0]`

C++: max priority queue

- `#include <queue>`
  `using namespace std;`
- Initialize
  - `priority_queue<int> pq;`
- Insert x
  - `pq.push(x)`
- Delete the maximum
  - `pq.pop()`
- Get the maximum
  - `pq.top()`